# CS 0007: Introduction to Java Lecture 15

Nathan Ong
University of Pittsburgh
November 1, 2016

# Main Portions of OOP

- Building a class
- Determining inter-class relationships
  - Subclasses
  - Superclasses
  - Ownership

# Building Classes

There are several parts of a class, much like there are several parts of an object

- Properties
  - Static: Class-level variables
  - Non-static: Instance variables
- Methods (originally functions)
  - Static: Class methods
  - Non-static: Instance methods
  - Special: Constructor

# Properties

- Inherent things (objects/primitives) describing the class or instance
- Placed within the class, but outside any method
- Class-level variables
  - Static
  - Usually constants
  - Declared and assigned outside any object
- Instance variables
  - Non-static
  - Declared by each object construction, which allows them to contain different values

# Class-level Variables

- Constants (`final` keyword)
  - Usually refers to limits or specific values
  - Signified in ALL_CAPS_WITH_UNDERSCORES
  - Variable cannot be changed
  - Usually visible to anyone (`public` keyword)

```
public static final double PI = 3.14;

int radius = 5;
double area = Math.PI*radius*radius;
```

# Class-level Variables

- Non-Constants
  - Usually performs bookkeeping (e.g. how many objects did you instantiate?)
  - Usually visible only to the class (`private` keyword)

```
private static int nextID = 0;
```

# Instance Variables

- Variables that are object instance dependent
- All cars have a color, but not all cars are red
- Possibly constant (`final` keyword)
  - Must be assigned at construction
  - Cannot change value for the lifetime of the object

# Example

```
public class Car
{
      private final int idNum;
      private Color color;
      private String
  licensePlate;
      …
}
```

# How to Determine?

- Depends on your needs and requirements of the scenario
- A good rule of thumb: Restrict everything as much as possible. If a variable does not need to change, make it `final` to prevent possible accidents. If a variable does not need to be directly visible to everyone, make it `private`.

# Methods

- Functions that alter the object or operate using the properties of the object
- This allows further control over how the object should be used, essentially providing guidelines to programmers on correct usage
- Three main types
  - Static: Class methods
  - Non-static: Instance methods
  - Special: Constructor

# Constructors

- Essentially a method initializing the object
- Calling this special method requires the keyword **new**
- We have seen this before!!!
- But how do I make one?

# Constructor

**Function Header**

1. Visibility type (`public`/`protected`/`private`)
2. ~~`static` (For now, required)~~
3. ~~Return Type~~
4. ~~functionName~~ Same name as the class
5. Parentheses “()”
   - Parameters
   a) Type1 parameterName1
   b) Type2 parameterName2
   c) …

**Function Body**

6. Curly Brackets/Braces “{}”
   - ~~`return` a value~~

# Car Example

```
public class Car
{
  private final int idNum;
  private Color color;
  private String licensePlate;

  public Car(Color color)
  {
  //initialize instance variables
  }//end constructor(Color)
  …
}//End class Car
```

# Instance Variable Manipulation

- To reference non-static object-dependent variables, use the keyword `this` followed by the dot operator.
- The keyword can distinguish these object-dependent variables and parameters.
- It can be used regularly, with no need to declare them.

# Car Example

```
public class Car
{
     private final int idNum;
     private Color color;
     private String licensePlate;

     public Car(Color color)
     {
     this.color = color;
     …
     }//end constructor(Color)
     …
}//End class Car
```

# Control Question

- Should we allow the function caller (e.g. random user) to make the car's ID number?  Probably not.
- We want the ID number to be unique.
- Simple solution: just count up by one every time a new car is made.
- We can do this via a class-level variable.

# Car Example

```
public class Car
{
     private static int nextIDNum = 0;
     private final int idNum;
     private Color color;
     private String licensePlate;

     public Car(Color color)
     {
     this.color = color;
     this.idNum = Car.nextIDNum;
     Car.nextIDNum++;
     …
     }//end constructor(Color)
     …
}//End class Car
```

# Car Example

```
public Car(Color color)
{
  …
  this.idNum = Car.nextIDNum;
  Car.nextIDNum++;
  …
}//end constructor(Color)
```

Because nextIDNum is static, and was initialized to 0, the first car has an ID number of 0. Then the static value is increased by 1. The next time we construct a new car, its ID number will be 1.

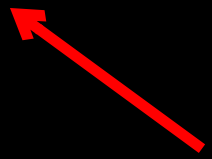
# Method Overloading

- Remember function overloading?
- This can apply to constructors as well!

# Overloading Constructors

```
public class Car
{
 …
 public Car(Color color)
 {
 this.color = color;
 this.idNum = Car.nextIDNum;
 Car.nextIDNum++;
 …
 }//end constructor(Color)

 //default color is Color.RED
 public Car()
 {
 this(Color.RED);
 }//end constructor()
 …
}//End class Car
```



Referencing other constructors can be done with the keyword `this`

# Overloading Constructors

- If you will call another constructor, then that is the only line of code that can be present in the constructor

# Functional Functions

- The most common functions are getters and setters
  - Getters retrieve the values of instance variables and return them
  - Setters alter the values of the instance variables
- Why use these functions rather than setting the variable to be public?
- Control!

# Car Example

```
public class Car
{
     private static int nextIDNum = 0;
     private final int idNum;
     private Color color;
     private String licensePlate;

     public int getIDNum()
     {
     return this.idNum;
     }//end function()
     …
}//End class Car
```

# Car Example

```
public class Car
{
     private static int nextIDNum = 0;
     private final int idNum;
     private Color color;
     private String licensePlate;

     public void setColor(Color color)
     {
     this.color = color;
     }//end function(Color)
     …
}//End class Car
```

# Why Bother?

- Having these methods ensure you have full control over the instance variables
- Should you decide to change how the variable is accessed or altered, you only need to change the method.
- You only add them when you need them!

# Recap

A class needs these components:

- Class-level/Instance Variables
- Methods
- Constructor(s)